



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



LAB MANUAL

Topic: Dictionaries in Python

Subject: Python Programming

Objective:

The objective of this lab manual is to help students understand Dictionaries in Python. After completing this lab, students will be able to create dictionaries, access dictionary values, update dictionary data, remove items, use dictionary functions, loop through dictionaries, and solve basic programming problems using dictionaries.

Background:

Python provides different data structures to store data. One of the most useful data structures is the Dictionary.

A dictionary is used to store data in key-value pairs. Each value in a dictionary is connected with a unique key. Instead of accessing data by index numbers, dictionaries allow us to access data using meaningful keys.

Example:

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "Artificial Intelligence"  
}
```

In this example:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



"name" is a key and "Ali" is its value.

"age" is a key and 20 is its value.

"department" is a key and "Artificial Intelligence" is its value.

Dictionaries are very useful when we want to store structured data such as student records, employee details, product information, marks records, and user profiles.

Important Features of Dictionaries:

Dictionaries store data in key-value pairs.

Dictionaries are ordered.

Dictionaries are mutable, which means values can be changed.

Dictionaries do not allow duplicate keys.

Dictionary keys must be unique.

Dictionary values can be of any data type.

Required Software:

Python 3

VS Code, PyCharm, Jupyter Notebook, or any Python IDE

Basic Syntax of Dictionary:

```
dictionary_name = {  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3"  
}
```

Example:

```
student = {  
    "name": "Sara",  
    "age": 21,
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
"city": "Hyderabad"  
}
```

```
print(student)
```

Output:

```
{'name': 'Sara', 'age': 21, 'city': 'Hyderabad'}
```

Common Dictionary Functions and Methods:

Function / Method	Description
get()	Returns the value of a specific key
keys()	Returns all keys from the dictionary
values()	Returns all values from the dictionary
items()	Returns all key-value pairs
update()	Updates dictionary with new or existing data
pop()	Removes an item using its key
popitem()	Removes the last inserted item



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Function / Method	Description
<code>clear()</code>	Removes all items from the dictionary
<code>copy()</code>	Creates a copy of the dictionary
<code>fromkeys()</code>	Creates a dictionary with specified keys and values

Example 1: Creating a Dictionary

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "Artificial Intelligence"  
}
```

`print(student)`

Output:

None

```
{'name': 'Ali', 'age': 20, 'department': 'Artificial Intelligence'}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Explanation:

In this example, a dictionary named student is created. It stores student information in key-value pairs.

Example 2: Accessing Dictionary Values

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "Artificial Intelligence"  
}
```

Python

```
print("Name:", student["name"])  
print("Age:", student["age"])  
print("Department:", student["department"])
```

Output:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



None

Name: Ali

Age: 20

Department: Artificial Intelligence

Explanation:

Dictionary values can be accessed by using their keys inside square brackets.

Example 3: Using get() Function

Code:

Python

```
student = {  
    "name": "Sara",  
    "age": 21,  
    "city": "Hyderabad"  
}
```

Python

```
print(student.get("name"))  
print(student.get("city"))
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print(student.get("department"))
```

Output:

None

Sara

Hyderabad

None

Explanation:

The `get()` function is used to access dictionary values safely. If the key does not exist, it returns `None` instead of generating an error.

Example 4: Adding New Item in Dictionary

Code:

Python

```
student = {  
    "name": "Ahmed",  
    "age": 19  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
student["department"] = "Computer Science"
```

Python

```
print(student)
```

Output:

None

```
{'name': 'Ahmed', 'age': 19, 'department': 'Computer Science'}
```

Explanation:

A new key-value pair can be added by assigning a value to a new key.

Example 5: Updating Dictionary Value

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,
```




**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
"city": "Karachi"  
}
```

Python

```
student["age"] = 21  
  
student["city"] = "Hyderabad"
```

Python

```
print(student)
```

Output:

None

```
{'name': 'Ali', 'age': 21, 'city': 'Hyderabad'}
```

Explanation:

Dictionaries are mutable, so their values can be changed after creation.

Example 6: Using update() Function

Code:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
student = {  
    "name": "Zain",  
    "age": 22  
}
```

Python

```
student.update({"age": 23})  
student.update({"department": "AI"})
```

Python

```
print(student)
```

Output:

None

```
{'name': 'Zain', 'age': 23, 'department': 'AI'}
```

Explanation:

The update() function can update an existing value or add a new key-value pair.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Example 7: Removing Item Using pop()

Code:

Python

```
student = {  
    "name": "Sara",  
    "age": 21,  
    "city": "Hyderabad"  
}
```

Python

```
student.pop("age")
```

Python

```
print(student)
```

Output:

None

```
{'name': 'Sara', 'city': 'Hyderabad'}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Explanation:

The pop() function removes an item from the dictionary using its key.

Example 8: Removing Last Item Using popitem()

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "AI"  
}
```

Python

```
student.popitem()
```

Python

```
print(student)
```

Output:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



None

```
{'name': 'Ali', 'age': 20}
```

Explanation:

The `popitem()` function removes the last inserted key-value pair from the dictionary.

Example 9: Using `del` Keyword

Code:

Python

```
student = {  
    "name": "Ahmed",  
    "age": 19,  
    "city": "Karachi"  
}
```

Python

```
del student["city"]
```

Python

```
print(student)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Output:

None

```
{'name': 'Ahmed', 'age': 19}
```

Explanation:

The del keyword is used to remove a specific item from the dictionary.

Example 10: Using clear() Function

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20  
}
```

Python

```
student.clear()
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
print(student)
```

Output:

None

```
{}
```

Explanation:

The clear() function removes all items from the dictionary and returns an empty dictionary.

Example 11: Using keys() Function

Code:

Python

```
student = {  
    "name": "Sara",  
    "age": 21,  
    "city": "Hyderabad"  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
print(student.keys())
```

Output:

None

```
dict_keys(['name', 'age', 'city'])
```

Explanation:

The keys() function returns all keys available in the dictionary.

Example 12: Using values() Function

Code:

Python

```
student = {  
    "name": "Sara",  
    "age": 21,  
    "city": "Hyderabad"  
}
```




**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
print(student.values())
```

Output:

None

```
dict_values(['Sara', 21, 'Hyderabad'])
```

Explanation:

The values() function returns all values available in the dictionary.

Example 13: Using items() Function

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "AI"  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
print(student.items())
```

Output:

None

```
dict_items([('name', 'Ali'), ('age', 20), ('department', 'AI')])
```

Explanation:

The items() function returns all key-value pairs from the dictionary.

Example 14: Looping Through Dictionary Keys

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "AI"  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
for key in student:  
  
    print(key)
```

Output:

name
age
department

Explanation:

By default, a loop through a dictionary returns its keys.

Example 15: Looping Through Dictionary Values

Code:

Python

```
student = {  
  
    "name": "Sara",  
  
    "age": 21,  
  
    "city": "Hyderabad"  
  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
for value in student.values():  
    print(value)
```

Output:

Sara

21

Hyderabad

Explanation:

The values() function is used with a loop to access all dictionary values.

Example 16: Looping Through Key-Value Pairs

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "department": "AI"  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
for key, value in student.items():  
    print(key, ":", value)
```

Output:

name : Ali
age : 20
department : AI

Explanation:

The items() function is used with a loop to access both keys and values together.

Example 17: Checking Key in Dictionary

Code:

Python

```
student = {  
    "name": "Ahmed",  
    "age": 19  
}
```

Python

```
if "name" in student:
```



```
print("Name key exists")  
else:  
    print("Name key does not exist")
```

Output:

Name key exists

Explanation:

The in keyword is used to check whether a key exists in a dictionary or not.

Example 18: Copying a Dictionary

Code:

Python

```
student = {  
    "name": "Ali",  
    "age": 20  
}
```

Python

```
new_student = student.copy()
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print("Original Dictionary:", student)
print("Copied Dictionary:", new_student)
```

Output:

```
Original Dictionary: {'name': 'Ali', 'age': 20}
Copied Dictionary: {'name': 'Ali', 'age': 20}
```

Explanation:

The copy() function creates a copy of the dictionary.

Example 19: Nested Dictionary

Code:

Python

```
students = {

    "student1": {

        "name": "Ali",

        "age": 20

    },

    "student2": {

        "name": "Sara",

        "age": 21

    }

}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
}
```

Python

```
print(students["student1"]["name"])  
  
print(students["student2"]["age"])
```

Output:

Ali
21

Explanation:

A nested dictionary is a dictionary inside another dictionary. It is useful for storing multiple records.

Solved Program 1: Store and Display Student Information

Problem:

Write a Python program to store student name, age, department, and city in a dictionary and display all information.

Solution:

Python

```
student = {  
  
    "name": "Yousif",
```




Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



```
"age": 21,  
  
"department": "Artificial Intelligence",  
  
"city": "Hyderabad"  
  
}
```

Python

```
print("Student Name:", student["name"])  
  
print("Age:", student["age"])  
  
print("Department:", student["department"])  
  
print("City:", student["city"])
```

Output:

Student Name: Yousif

Age: 21

Department: Artificial Intelligence

City: Hyderabad

Solved Program 2: Add New Subject in Dictionary

Problem:

Write a Python program to create a dictionary of subjects and marks. Add one new subject and display the updated dictionary.

Solution:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
marks = {  
    "Python": 85,  
    "Database": 78,  
    "Mathematics": 90  
}
```

```
marks["AI"] = 88
```

```
print(marks)
```

Output:

```
{'Python': 85, 'Database': 78, 'Mathematics': 90, 'AI': 88}
```

Solved Program 3: Update Marks in Dictionary

Problem:

Write a Python program to update the marks of Python from 80 to 95.

Solution:

Python

```
marks = {  
    "Python": 80,
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
"Database": 75,
```

```
"AI": 85
```

```
}
```

```
marks["Python"] = 95
```

```
print(marks)
```

Output:

```
{'Python': 95, 'Database': 75, 'AI': 85}
```

Solved Program 4: Calculate Total Marks from Dictionary

Problem:

Write a Python program to store subject marks in a dictionary and calculate total marks.

Solution:

Python

```
marks = {
```

```
    "Python": 85,
```

```
    "Database": 80,
```

```
    "Mathematics": 90,
```

```
    "AI": 88
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
}
```

```
total = sum(marks.values())
```

```
print("Marks:", marks)
```

```
print("Total Marks:", total)
```

Output:

```
Marks: {'Python': 85, 'Database': 80, 'Mathematics': 90, 'AI': 88}
```

```
Total Marks: 343
```

Solved Program 5: Calculate Average Marks from Dictionary

Problem:

Write a Python program to calculate average marks from a dictionary.

Solution:

Python

```
marks = {  
    "Python": 85,  
    "Database": 75,  
    "Mathematics": 90,  
    "AI": 80  
}
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
total = sum(marks.values())  
average = total / len(marks)  
  
print("Total Marks:", total)  
print("Average Marks:", average)
```

Output:

Total Marks: 330
Average Marks: 82.5

Solved Program 6: Find Highest and Lowest Marks

Problem:

Write a Python program to find the highest and lowest marks from a dictionary.

Solution:

Python

```
marks = {  
    "Python": 85,  
    "Database": 75,  
    "Mathematics": 90,  
    "AI": 80  
}
```

```
highest = max(marks.values())  
lowest = min(marks.values())
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print("Highest Marks:", highest)
print("Lowest Marks:", lowest)
```

Output:

Highest Marks: 90
Lowest Marks: 75

Solved Program 7: Search Student by Key

Problem:

Write a Python program to check whether the key "department" exists in a student dictionary.

Solution:

Python

```
student = {
    "name": "Ali",
    "age": 20,
    "department": "AI"
}
```

```
if "department" in student:
    print("Department is available")
else:
    print("Department is not available")
```

Output:

Department is available



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Solved Program 8: Remove an Item from Dictionary

Problem:

Write a Python program to remove city from a student dictionary.

Solution:

Python

```
student = {  
    "name": "Sara",  
    "age": 21,  
    "city": "Hyderabad"  
}
```

```
student.pop("city")
```

```
print(student)
```

Output:

```
{'name': 'Sara', 'age': 21}
```

Solved Program 9: Print All Keys and Values Separately

Problem:

Write a Python program to print all keys and values from a dictionary separately.

Solution:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
student = {  
    "name": "Ahmed",  
    "age": 19,  
    "city": "Karachi"  
}
```

```
print("Keys:")  
for key in student.keys():  
    print(key)  
  
print("Values:")  
for value in student.values():  
    print(value)
```

Output:

```
Keys:  
name  
age  
city  
Values:  
Ahmed  
19  
Karachi
```

Solved Program 10: Store Multiple Students Using Nested Dictionary

Problem:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Write a Python program to store information of two students using a nested dictionary.

Solution:

Python

```
students = {  
    "student1": {  
        "name": "Ali",  
        "age": 20,  
        "department": "AI"  
    },  
    "student2": {  
        "name": "Sara",  
        "age": 21,  
        "department": "Computer Science"  
    }  
}
```

```
print("First Student Name:", students["student1"]["name"])  
print("Second Student Department:", students["student2"]["department"])
```

Output:

First Student Name: Ali

Second Student Department: Computer Science



Solved Program 11: Count Frequency of Words Using Dictionary

Problem:

Write a Python program to count how many times each word appears in a list.

Solution:

```
words = ["apple", "banana", "apple", "mango", "banana", "apple"]
```

```
frequency = {}
```

Python

```
for word in words:
    if word in frequency:
        frequency[word] = frequency[word] + 1
    else:
        frequency[word] = 1
```

```
print(frequency)
```

Output:

```
{'apple': 3, 'banana': 2, 'mango': 1}
```

Solved Program 12: Create Dictionary from Two Lists

Problem:

Write a Python program to create a dictionary using one list of keys and one list of values.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Solution:

```
keys = ["name", "age", "city"]  
values = ["Ali", 20, "Hyderabad"]  
  
student = dict(zip(keys, values))  
  
print(student)
```

Output:

```
{'name': 'Ali', 'age': 20, 'city': 'Hyderabad'}
```

Practice Tasks:

Task 1:

Create a dictionary to store your name, age, city, and department. Print the complete dictionary.

Task 2:

Create a dictionary of three subjects and their marks. Print only the marks of one subject.

Task 3:

Create a student dictionary and add a new key named "semester".

Task 4:

Create a dictionary of subject marks and update the marks of one subject.

Task 5:

Create a dictionary and remove one item using pop().

Task 6:

Create a dictionary and print all keys using the keys() function.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Task 7:

Create a dictionary and print all values using the values() function.

Task 8:

Create a dictionary and print all key-value pairs using the items() function.

Task 9:

Create a dictionary of five products with their prices. Find the total price of all products.

Task 10:

Create a dictionary of student marks and find the highest and lowest marks.

Task 11:

Create a nested dictionary to store information of three students.

Task 12:

Create a list of repeated numbers and count the frequency of each number using a dictionary.

Task 13:

Create two lists: one for student names and one for marks. Convert them into a dictionary.

Task 14:

Create a dictionary and check whether a specific key exists in it.

Task 15:

Create a dictionary and clear all items using the clear() function.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Conclusion:

In this lab, we learned about Dictionaries in Python. A dictionary stores data in key-value pairs and is useful for structured data. Dictionaries are mutable, ordered, and do not allow duplicate keys. We practiced important dictionary functions such as `get()`, `keys()`, `values()`, `items()`, `update()`, `pop()`, `popitem()`, `clear()`, and `copy()`. We also learned how to loop through dictionaries, use nested dictionaries, and solve basic programming problems using dictionaries.